Experiment 1: Configuration of GPIO Pin and LED Blinking using STM32

A beginner-friendly practical guide to controlling hardware peripherals on STM32 microcontrollers

Made with GAMMA

Aim 🎯

Configure GPIO Pin

Set up GPIO pin PA5 as a digital output on the STM32 microcontroller

Control LED

Connect and control an LED through software programming

Verify Blinking

Test LED blinking functionality using software delay functions in STM32CubeIDE

Made with GAMMA

Apparatus / Requirements 🛠️

1

STM32 Nucleo-F446RE Development Board

Microcontroller board featuring STM32F446RE chip with built-in LED

2

USB Type A to Mini B Cable

For connecting the development board to your computer

3

Laptop/PC

Running Windows, Linux, or macOS operating system

4

STM32CubeIDE

Integrated development environment for STM32 programming

5

STM32CubeMX

Graphical configuration tool for generating initialization code

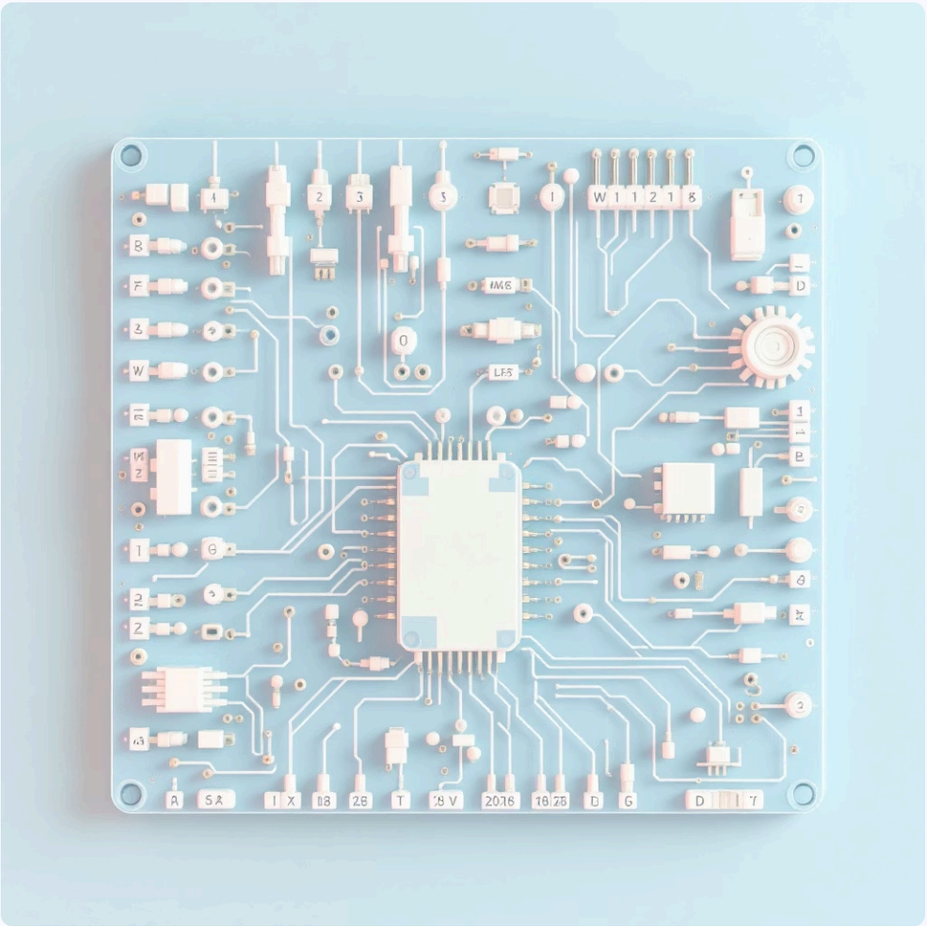
Made with GAMMA

Theory 📘

What is GPIO?

GPIO (General Purpose Input Output) pins are fundamental interfaces in microcontrollers that enable communication with external electronic components such as LEDs, switches, sensors, and displays. These versatile pins can be dynamically configured as either inputs to read data from external devices or as outputs to control external hardware.

In STM32 microcontrollers, GPIO pins are controlled through register programming or using HAL (Hardware Abstraction Layer) libraries. The HAL library provides user-friendly functions that abstract the complexity of direct register manipulation, making programming more accessible for beginners.



Register Programming

Direct manipulation of hardware registers for precise control

HAL Libraries

Pre-built functions that simplify hardware configuration and control

Built-in LED

Nucleo board has LED connected to PA5 pin for easy testing

Made with GAMMA

Procedure

Follow these step-by-step instructions to configure your STM32 board and program LED blinking:

Launch STM32CubeMX
Open the graphical configuration software on your computer

Select MCU
Choose STM32F446RE microcontroller from the device list

Configure PA5 Pin
Set pin PA5 as GPIO Output mode in the graphical interface

System Clock
Configure clock settings for optimal performance

Generate Code
Click "Generate Code" button to create project files

Open in CubeIDE
Launch STM32CubeIDE and import the generated project

Edit main.c
Locate and open the main.c file for code modification

Write Code
Add LED toggle code inside the infinite while loop

Build Program
Compile the code to check for errors and generate binary

Connect Board
Use USB cable to connect STM32 board to PC

Flash & Run
Upload the program to the microcontroller and execute

Observe LED
Watch the LED blinking pattern on the board

Made with GAMMA

Program / Code 💻

The core LED blinking functionality is implemented using two HAL library functions within an infinite loop:

```
while(1)
{
  HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
  HAL_Delay(500);
}
```

1

Toggle Pin

HAL_GPIO_TogglePin() function switches the pin state between HIGH and LOW

2

Delay 500ms

HAL_Delay() function pauses execution for 500 milliseconds

3

Repeat Forever

Infinite loop continuously repeats the toggle-delay sequence

This simple code creates a blinking LED with a period of 1 second (500ms on + 500ms off). The toggle function alternates the pin state each time it executes, while the delay controls the timing between state changes.

Made with GAMMA

Observation 👀

Testing different delay values reveals how timing affects LED visibility:

S. No.	Delay (ms)	LED Behavior	Observation
1	100	Very fast blink	Hard to notice individual blinks
2	200	Fast blinking	Visible flickering effect
3	500	Moderate blinking	Clearly visible blink pattern
4	1000	Slow blinking	Easy to observe and count

Pro Tip: The human eye can perceive blinking up to approximately 10-12 Hz (100ms delay). Beyond this frequency, LEDs appear continuously lit due to persistence of vision.

Made with GAMMA

Result 📊

100%

GPIO Configuration

Pin PA5 successfully configured as output

✓

LED Control

Blinking achieved with software delay

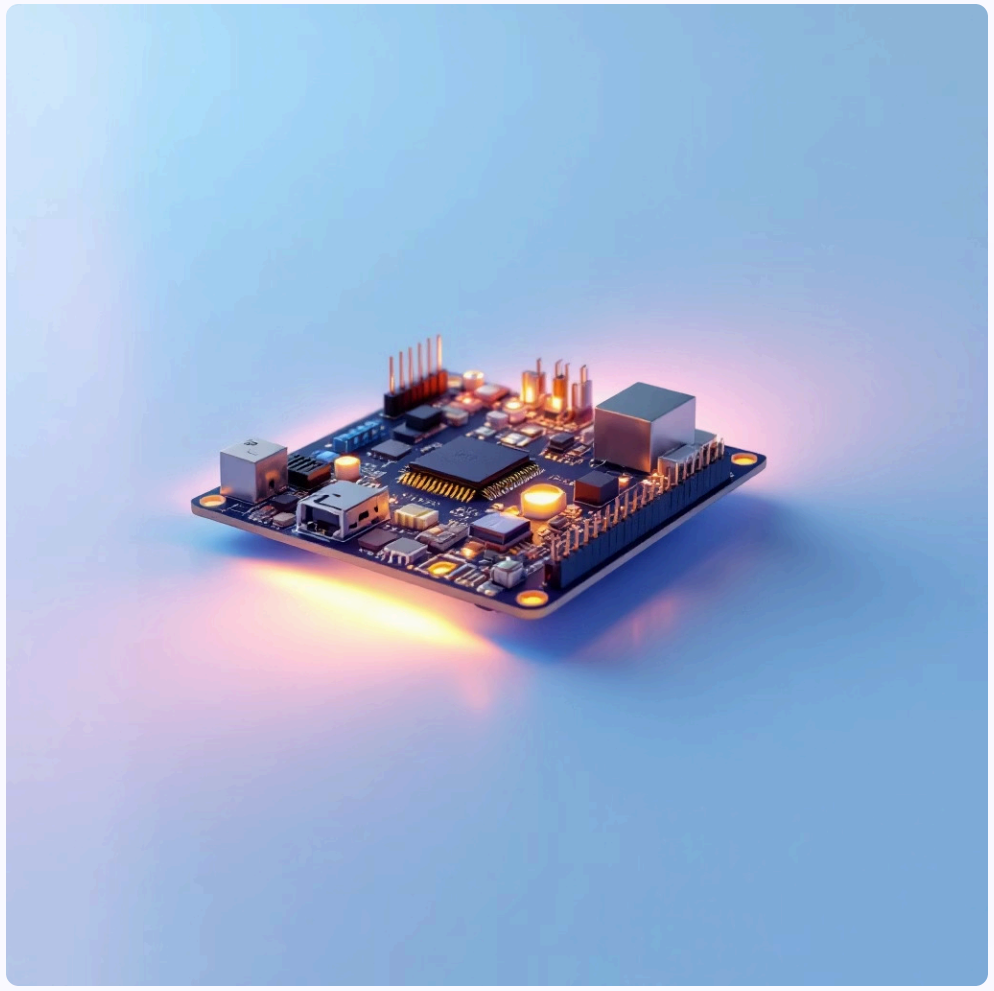
3+

Timing Variations

Multiple delay values tested successfully

Key Achievements

- Successful GPIO pin configuration using STM32CubeMX
- LED blinking implemented with HAL library functions
- Timing control demonstrated through variable delays
- Hardware-software integration verified



Made with GAMMA

Conclusion 💡

GPIO Configuration

Successfully learned how to configure GPIO pins as outputs in STM32 microcontrollers using graphical tools and HAL libraries

Hardware Control

Demonstrated practical control of external devices (LED) through software programming and pin manipulation

HAL Simplification

HAL libraries significantly reduced programming complexity compared to direct register manipulation

Timing Control

Delay functions provided precise control over LED blinking speed and timing patterns

This experiment provided a fundamental introduction to embedded system programming on STM32 platforms. The combination of STM32CubeMX for configuration and STM32CubeIDE for coding creates an efficient development workflow. Understanding GPIO control is essential for all future embedded projects involving sensors, displays, communication interfaces, and other peripherals.

Made with GAMMA

Learning Outcome 🧠

1

GPIO Configuration in STM32

Mastered the process of configuring GPIO pins using STM32CubeMX graphical interface and understanding pin modes, speed settings, and pull-up/pull-down resistors for reliable hardware interfacing

2

STM32CubeMX and CubeIDE Workflow

Understood the complete development workflow from initial hardware configuration in CubeMX to code generation, project import, and debugging in STM32CubeIDE for efficient embedded development

3

HAL Functions Knowledge

Gained practical knowledge of Hardware Abstraction Layer functions including HAL_GPIO_TogglePin() for pin control and HAL_Delay() for timing operations

4

Timing Control with Delays

Learned how to control program timing using delay functions and understand their impact on hardware behavior and system responsiveness in embedded applications

---

These foundational skills serve as building blocks for more advanced embedded system projects including sensor interfacing, communication protocols (UART, I2C, SPI), display drivers, and real-time operating systems.

Made with GAMMA